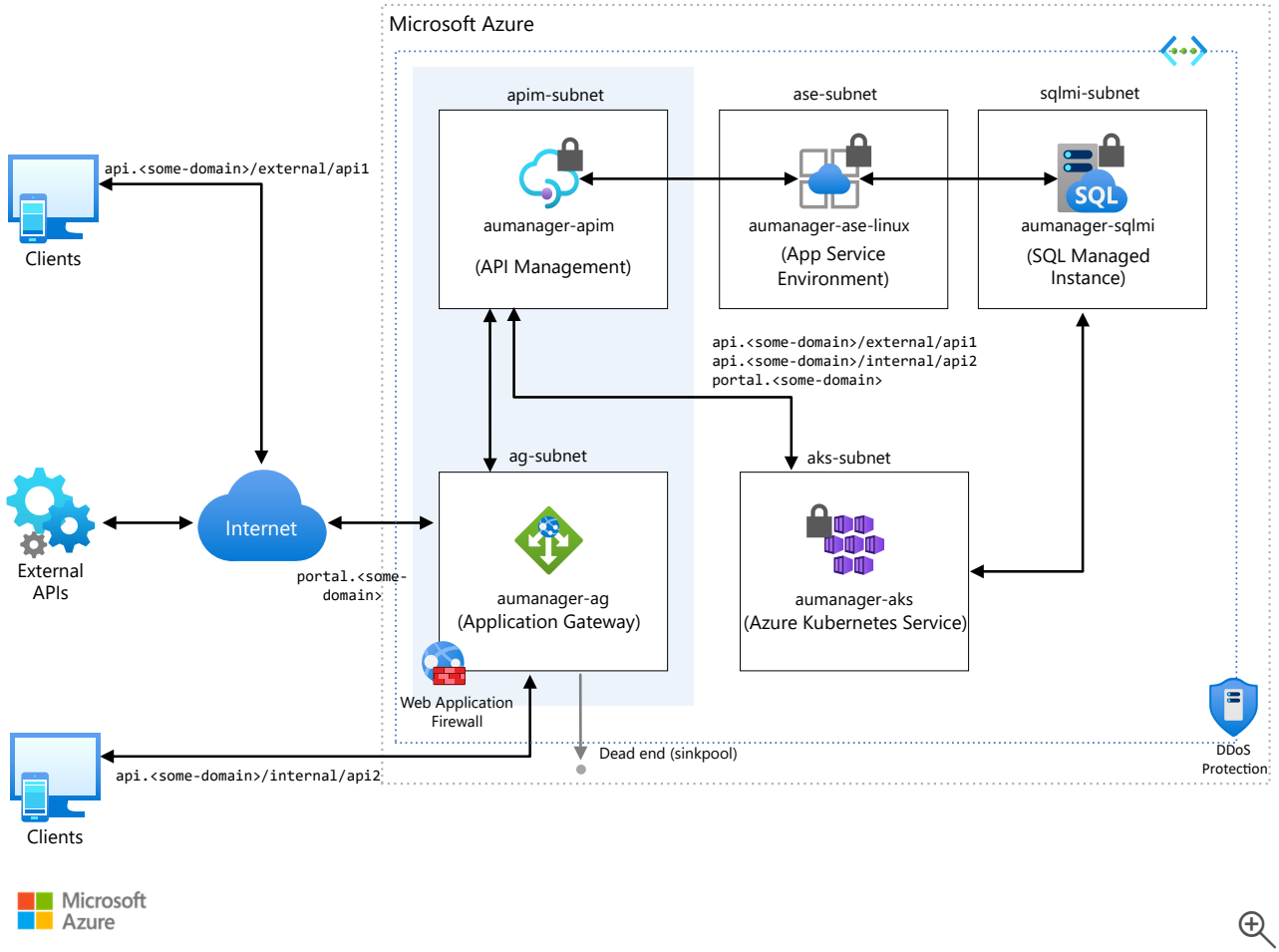
Protect APIs by using Application Gateway and API Management

Azure API Management | Azure Application Gateway

Organizations increasingly adopt API-first design approaches while facing growing threats to web applications. You need a comprehensive security strategy to protect APIs, especially when exposing AI-powered APIs and implementing Zero Trust architecture principles. The Gateway Routing pattern provides one approach to API security by protecting network traffic. The gateway restricts traffic source locations and traffic quality while supporting flexible routing rules. This article describes how to use Azure Application Gateway and Azure API Management to protect API access.

Architecture

This article doesn't address the application's underlying platforms, such as App Service Environment, Azure SQL Managed Instance, and Azure Kubernetes Service (AKS). Those parts of the diagram showcase what you can implement as a broader solution. This article specifically discusses the shaded areas, API Management, and Application Gateway.



Microsoft Azure

Download a Visio file of this architecture.

Workflow

1. Application Gateway receives HTTPS requests that the subnet's network security group (NSG) allows.

2. The web application firewall (WAF) on Application Gateway checks the request against WAF rules, including geomatch custom rules. If the request is valid, the request proceeds.

3. Application Gateway sets up a URL proxy mechanism that sends the request to the proper back-end pool. The routing behavior depends on the URL format of the API call:

   - URLs formatted as `api.<some-domain>/external/*` can reach the back end to interact with the requested APIs.

   - Calls formatted as `api.<some-domain>/*` go to a dead end, called a *sinkpool*, which is a back-end pool with no target.

   - A routing rule at the Application Gateway level redirects users under `portal.<some-domain>/*` to the developer portal. Developers can manage APIs and their configurations from both internal and external environments. Alternatively, you can block the developer portal completely.

4. Application Gateway accepts and proxies internal calls from resources in the same Azure virtual network under `api.<some-domain>/internal/*`.

5. At the API Management level, APIs accept calls under the following patterns:

   - `api.<some-domain>/external/*`
   - `api.<some-domain>/internal/*`

   In this scenario, API Management uses public and private IP addresses. Public IP addresses support management operations on port 3443 for the management plane and for runtime API traffic in external virtual network configurations. When API Management sends a request to a public internet-facing back end, it shows a public IP address as the origin of the request. For more information, see IP addresses of API Management in a virtual network.

Components

- Azure Virtual Network enables many types of Azure resources to communicate privately with each other, the internet, and on-premises networks. In this architecture, Application Gateway tunnels public internet traffic into this private network.

- Application Gateway is a web traffic load balancer that manages traffic to web applications. This type of routing is known as *application layer (OSI Layer 7) load balancing*. In this architecture, the gateway provides routing and hosts a WAF to protect against common web-based attack vectors.

- API Management is a hybrid, multicloud management platform for APIs across all environments. API Management creates consistent, modern API gateways for existing back-end services. In this architecture, API Management operates in a fully private mode to offload cross-cutting concerns from the API code and hosts.

Alternatives

You can use other services to deliver a similar level of firewall and WAF protection:

- Azure Front Door provides built-in distributed denial-of-service (DDoS) protection and global load balancing.

- Azure Firewall provides network-level protection and centralized security policy management.

- Partner solutions, such as Barracuda CloudGen WAF for Azure , or other WAF solutions are available in the Microsoft Marketplace .

Recommendations

This architecture focuses on implementing the whole solution and testing API access from inside and outside the API Management virtual network. For more information about the integration process, see Integrate API Management in an internal virtual network by using Application Gateway.

To communicate with private resources in the back end, place Application Gateway and API Management in the same virtual network as the resources or in a peered virtual network.

- The private, internal deployment model allows API Management to connect to an existing virtual network, which makes it reachable from inside that network context. To enable this feature, deploy either the **Developer** or **Premium** API Management tiers for classic virtual network injection. For newer virtual network options, use the **Standard v2** or **Premium v2** tiers with virtual network integration or injection capabilities.

- If your clients operate in a different subscription or are managed with a different Microsoft Entra ID directory, use Azure Private Link for Application Gateway to provide private connectivity to Application Gateway from client virtual networks across subscriptions and regions.

- Manage Application Gateway certificates in Azure Key Vault.
- To personalize interactions with the services, you can use canonical name (CNAME) entries.

Considerations

These considerations implement the pillars of the Azure Well-Architected Framework, which is a set of guiding tenets that you can use to improve the quality of a workload. For more information, see Well-Architected Framework.

Reliability

Reliability helps ensure that your application can meet the commitments that you make to your customers. For more information, see Design review checklist for Reliability.

Application Gateway always deploys in a highly available configuration, regardless of the instance count. To reduce the impact of a zone malfunction, you can configure the application gateway to span multiple availability zones. For more information, see Autoscaling and high availability.

Enable zone redundancy for your API Management service components to provide resiliency and high availability. Zone redundancy replicates the API Management gateway and control plane across datacenters in physically separated zones. This configuration makes them resilient to zone failure. You must use the API Management **Premium** tier to support availability zones.

API Management also supports multiregion deployments, which can improve availability if one region goes offline. For more information, see Multiregion support. In this topology, deploy one application gateway for each region because Application Gateway is a regional service.

Security

Security provides assurances against deliberate attacks and the misuse of your valuable data and systems. For more information, see Design review checklist for Security.

For more information about Application Gateway security, see Azure security baseline for Application Gateway.

For more information about API Management security, see Azure security baseline for API Management.

Always implement the following security measures:

- Use Azure Web Application Firewall policies with the latest Open Web Application Security Project (OWASP) Core Rule Set (CRS) 3.2 or newer to protect against common web vulnerabilities, including the OWASP Top 10 threats.

- Configure WAF geomatch custom rules to block or allow traffic based on geographic location. This approach provides some protection against DDoS attacks.
- Enable application (Layer 7) DDoS protection by using Azure Web Application Firewall with Application Gateway to protect against volumetric and protocol-based attacks. Combine Azure DDoS Protection with application-design practices to enhance DDoS mitigation features.
- Use private endpoints for API Management to provide secure inbound connectivity.
- Enable Microsoft Defender for APIs to monitor API security posture and detect threats.
- Configure WAF bot protection rules to identify and block malicious bots.

Cost Optimization

Cost Optimization focuses on ways to reduce unnecessary expenses and improve operational efficiencies. For more information, see Design review checklist for Cost Optimization.

The cost of this architecture depends on several configuration aspects:

- **Service tiers:** Consider Standard v2 and Premium v2 tiers for API Management to improve cost efficiency and performance.
- **Scalability:** Services dynamically allocate the number of instances to support a given demand.
- **Runtime duration:** Costs vary depending on whether the architecture runs continuously or only a few hours every month.
- **Data transfer:** Multiregion deployments incur transfer costs between regions.
- **WAF processing:** Costs depend on the number of requests and rules evaluated.

Consider the following cost optimization strategies:

- Use the API Management consumption tier for low usage, variable workloads where you pay only for actual usage.
- Implement Application Gateway autoscaling to optimize instance counts based on demand.

After you assess these aspects, use the Azure pricing calculator to estimate pricing.

Operational Excellence

Operational Excellence covers the operations processes that deploy an application and keep it running in production. For more information, see Design review checklist for Operational Excellence.

Implement comprehensive monitoring and observability:

- Configure API Management diagnostics to send logs to Azure Monitor so that you can use Log Analytics for detailed API analytics.
- Set up Application Gateway diagnostics to monitor WAF events and performance metrics.
- Implement API Management alerts for API performance and availability thresholds.

Performance Efficiency

Performance Efficiency refers to your workload's ability to scale to meet user demands efficiently. For more information, see Design review checklist for Performance Efficiency.

Application Gateway serves as the entry point for this architecture, and the Azure Web Application Firewall feature requires processing power for each request analysis. To allow Application Gateway to expand its computational capacity on demand, enable autoscaling. For more information, see Autoscaling and zone redundancy in Application Gateway. Follow the product documentation recommendations for Application Gateway infrastructure configuration, including proper subnet sizing. This approach ensures the subnet is large enough to support full scale-out.

Consider the following performance optimizations for API Management:

- Enable API Management autoscaling to automatically respond to increasing request volumes.
- Use API Management caching policies to reduce back-end load and improve response times.
- Implement API Management rate limiting to protect back-end services from excessive load.
- Use Standard v2 or Premium v2 tiers to improve performance and networking capabilities.

Next steps

To design APIs, follow good web API design guidelines. To implement APIs, use good web API implementation practices.

Related resources

- Gateway Routing pattern: Route requests to multiple services by using a single endpoint.
- Gateway Aggregation pattern: Aggregate multiple requests into a single request.
- Gateway Offloading pattern: Offload shared functionality to an API gateway.
- URL path-based routing overview
- Tutorial: Create an application gateway with URL path-based redirection by using the Azure CLI